School of Information Technologies and Engineering, ADA University

CSCI3613 – Artificial Intelligence

Spring 2026

Course Project Report

Team 7

	Full Name	Student ID
Member 1	Lala Huseynova	18047
Member 2	Ilknur Huseynova	15340
Member 3	Ibrahim Najafguliyev	19888

Project – Nardy AI

Problem formulation

In Nardy (Long Backgammon) each of the two players move 15 checkers around 24-point board and try to bear them all off first. Every turn player tosses 2 dices randomly thus neither of the players know what moves will be available for the next turn. This randomness is what makes this problem interesting – constructing such a strategy which takes into account the possible moves of the opponent for all dice outcomes that might occur.

Our goal was to build an AI agent that can play Nardy against a human player at a reasonable level. The agent must select the best move sequence given an unknown dice outcome, at the same time considering the opponent's possible response.

EDA and data preprocessing

In Nardy we don't have any external dataset. When the dices are rolled, the positions are fully determined. That's why instead of preprocessing data; we analyzed the structure of the game to understand what AI handle should.

Representation of the board: We store the board as an array of 24 integers. The positive values are representing player 1 checkers; negative values are representing player 2 checkers. One number store both the piece type and count, which makes generating moves simpler.

Dice distribution: There are 21 possible distinct dice outcomes from which 6 of them are doubles with the chances of 1/36 and rest 15 non double pairs with 2/36 chances each. Every possible dice outcome is checked and not sampled which means the AI's estimates are exact and not approximated.

Head factor: as a rule, only one checker can leave the head per turn. We tested this rule in isolation before wiring it into the move generator.

Implementation

Python handles the game's logic, and JavaScript shows the board in the browser. The project supports Human vs Human and Human vs AI modes, with separate Flask apps for each mode. The core game rules are implemented in game_logic.py, while the AI decision-making is implemented in ai_agent.py. The system is implemented as a Flask application with a JavaScript front end and Python game logic.

Algorithms Applied and Why

For the AI player, we applied the Expectiminimax algorithm. This algorithm is actually the most suitable one because Nardy includes both strategy and randomness. Expectiminimax handles this

randomness by combining decision nodes, where the AI chooses the best move, with chance nodes, where possible dice outcomes are averaged using their probabilities.

We also implemented a Random AI agent for comparison. The Random AI selects a legal move randomly, so it helps us to create some sort of comparison to show whether the Expectiminimax AI performs better than random play.

Design Decisions

We separated the project into clear modules:

- game_state.py - this stores the board state.
- game_logic.py – this file checks legal moves and applies rules.
- ai_agent.py - contains the Expectiminimax AI.
- random_ai.py - contains the Random AI.
- compare_ai_agents.py – this file runs automatic AI-vs-random simulations.
- app.py - runs the human-vs-human Flask version of the game.
- app_ai.py - runs the human-vs-AI Flask version of the game.
- run_game.py - launches the project and lets the user choose between Human mode and AI mode.

This modular design keeps the rules, state, AI, and evaluation separate, making the system easier to test, debug, and extend.

The AI uses depth-limited search because a full game-tree search would be computationally too expensive. We used depth 2 or 3 depending on performance constraints.

To evaluate non-terminal game states, we implemented a **heuristic evaluation function**. This function estimates how good a board position is when the search depth limit is reached. The heuristic considers:

- **Pip count** (distance of checkers from bearing off)
- **Borne-off checkers** (progress toward winning)
- **Blots** (single exposed checkers, which are risky)
- **Blocks** (points occupied by two or more checkers, which provide control)
- **Home-board checkers** (progress toward endgame and bearing off)

These features are combined into a weighted score that allows the AI to compare board states and choose stronger moves even without exploring the full game.

Evaluation

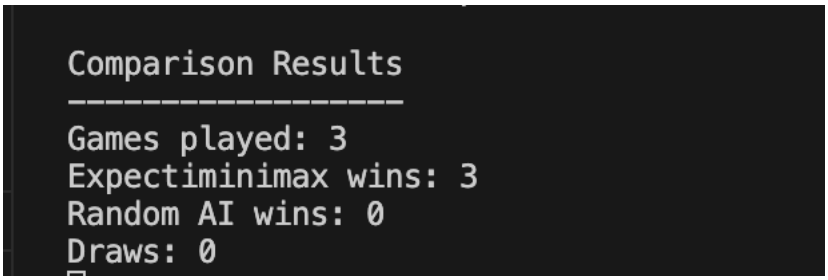
We evaluated the algorithm by running automatic games between the Expectiminimax AI and the Random AI. Both agents used the same legal move generator and dice rules, but Random AI chose moves randomly while Expectiminimax searched ahead and evaluated future outcomes.

For short tests, we used fewer turns to verify that both agents played correctly. For final evaluation, we increased the number of turns and compared win rate, borne-off checkers, and board evaluation score.

Experiments

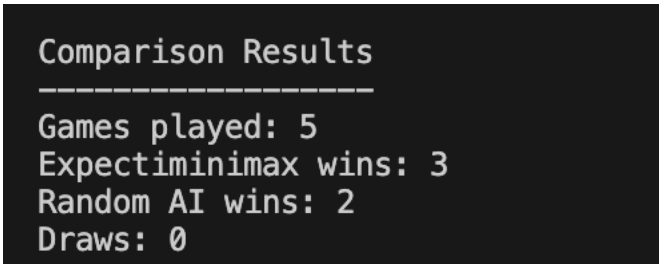
We evaluated the performance of the Expectiminimax AI by running automatic games against the Random AI using compare_ai_agents.py. In this experiment, we simulated 3 full games, each limited to a maximum of 150 turns. Both agents used the same legal move generator and dice rules.

The Expectiminimax AI selected moves using depth-limited search with a heuristic evaluation function, while the Random AI selected moves randomly. The experiment recorded the winner of each game.



Notes: We used search depth 2 to keep the AI fast while still making good decisions. Depth 3 could be more accurate, but it is slower. We ran 3 games with a maximum of 150 turns to keep the experiments quick and manageable.

We tested the Expectiminimax AI against the Random AI using compare_ai_agents.py. We ran 5 games, with a maximum of 1350 turns per game. Both agents used the same game rules.
The Expectiminimax AI used search and a heuristic function to choose moves, while the Random AI chose moves randomly.



Discussion of results

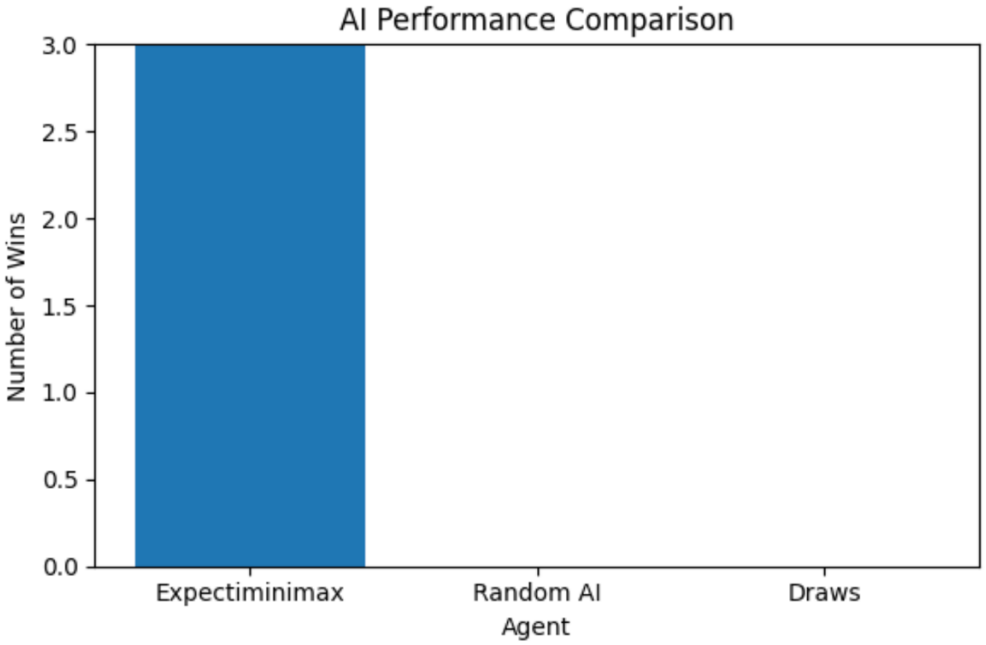
The first experiment used 3 games with a maximum of 150 turns. In that test, the Expectiminimax AI won 3 out of 3 games, so it clearly performed better than the Random AI. This showed that the AI's search-based decision-making and heuristic evaluation helped it choose stronger moves.

Then, we ran a larger experiment with 5 games and a maximum of 1350 turns. In this test, Expectiminimax won 3 games, Random AI won 2 games, and there were 0 draws. This result is more balanced because more games and more turns give the Random AI more chances to benefit from dice randomness.

Overall, the results show that Expectiminimax performs better than Random AI, but dice rolls still affect the outcome. More experiments would make the results more reliable.

The bar chart shows that Expectiminimax performed better than the Random AI.

Expectiminimax 3 – 0 Random AI agent



Expectiminimax 3 – 2 Random AI agent